

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

df_train = pd.read_csv('testset.csv')
df_test = pd.read_csv('testset.csv')

train_prices = df_train['Close'].values.reshape(-1, 1)
test_prices = df_test['Close'].values.reshape(-1, 1)

scaler = MinMaxScaler()
scaled_train = scaler.fit_transform(train_prices)
scaled_test = scaler.transform(test_prices)

def create_sequences(data, seq_length):
    x = []
    y = []
    for i in range(len(data) - seq_length):
        x.append(data[i:i+seq_length])
        y.append(data[i+seq_length])
    return np.array(x), np.array(y)

seq_length = 60
x_train, y_train = create_sequences(scaled_train, seq_length)
x_test, y_test = create_sequences(scaled_test, seq_length)

x_train.shape, y_train.shape, x_test.shape, y_test.shape
((65, 60, 1), (65, 1), (65, 60, 1), (65, 1))

x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
x_test_tensor = torch.tensor(x_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

train_dataset = TensorDataset(x_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

## Step 2: Define RNN Model
class RNNModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, num_layers=1,
output_size=1):
        super(RNNModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        # Define the RNN layer

```

```

        self.rnn = nn.RNN(input_size, hidden_size, num_layers,
batch_first=True)
        # Define the final fully connected layer
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # Initialize hidden state
        h0 = torch.zeros(self.num_layers, x.size(0),
self.hidden_size).to(x.device)
        # Forward propagate RNN
        out, _ = self.rnn(x, h0)
        # Extract the output of the last time step
        out = self.fc(out[:, -1, :])
        return out

model = RNNModel()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

print(device)

cpu

!pip install torchinfo

Requirement already satisfied: torchinfo in c:\users\kishore\
anaconda3\envs\torch_env\lib\site-packages (1.8.0)

from torchinfo import summary

# input_size = (batch_size, seq_len, input_size)
summary(model, input_size=(64, 60, 1))

=====
=====
Layer (type:depth-idx)          Output Shape
Param #
=====
=====
RNNModel                        [64, 1]          --
├─RNN: 1-1                      [64, 60, 32]
1,120
├─Linear: 1-2                   [64, 1]          33
=====
=====
Total params: 1,153
Trainable params: 1,153
Non-trainable params: 0
Total mult-adds (M): 4.30
=====
=====

```

```
Input size (MB): 0.02
Forward/backward pass size (MB): 0.98
Params size (MB): 0.00
Estimated Total Size (MB): 1.00
=====
=====
```

```
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
epochs = 50
```

```
train_losses = []
```

```
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, targets)

        # Backward pass and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)

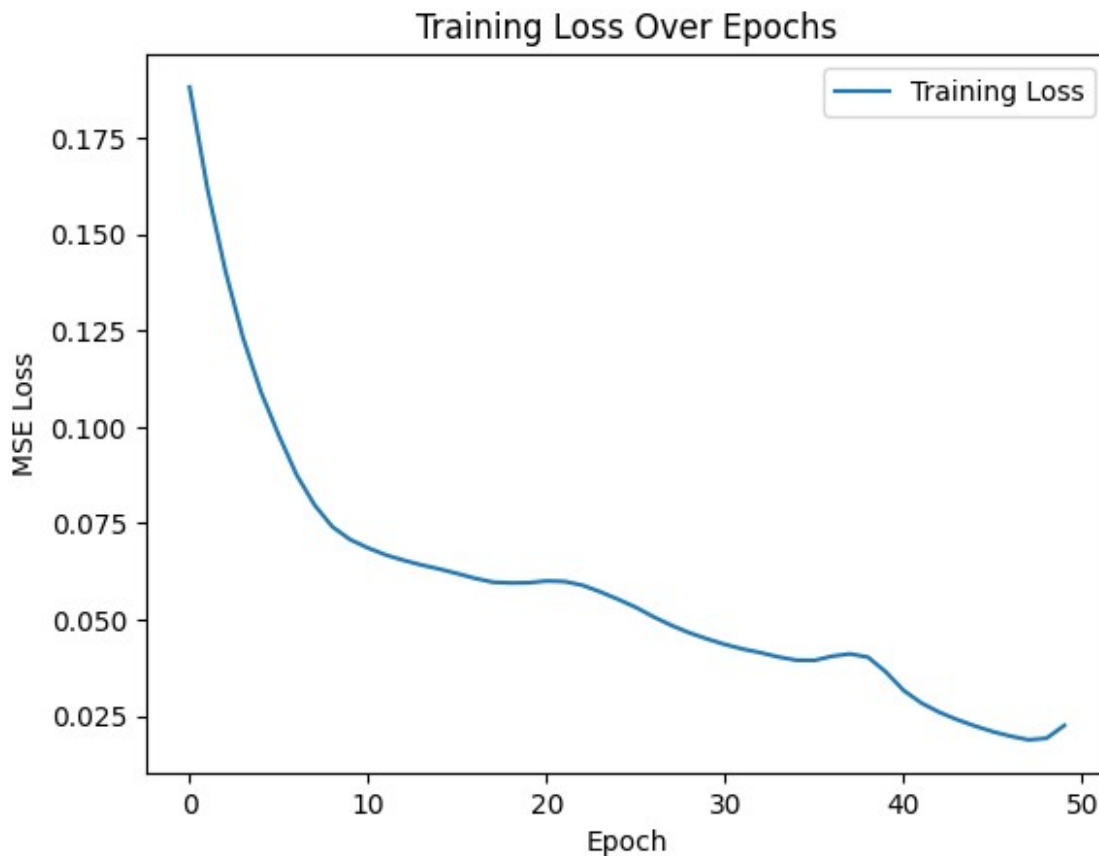
    epoch_loss = running_loss / len(train_loader.dataset)
    train_losses.append(epoch_loss)

    if (epoch+1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss:.4f}')
```

```
Epoch [10/50], Loss: 0.0708
Epoch [20/50], Loss: 0.0596
Epoch [30/50], Loss: 0.0450
Epoch [40/50], Loss: 0.0365
Epoch [50/50], Loss: 0.0226
```

```
print('Name: SARAVANAN P')
print('Register Number: 212224230253')
plt.plot(train_losses, label='Training Loss')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Training Loss Over Epochs')
plt.legend()
plt.show()
```

Name: SARAIVANAN P
Register Number: 212224230253



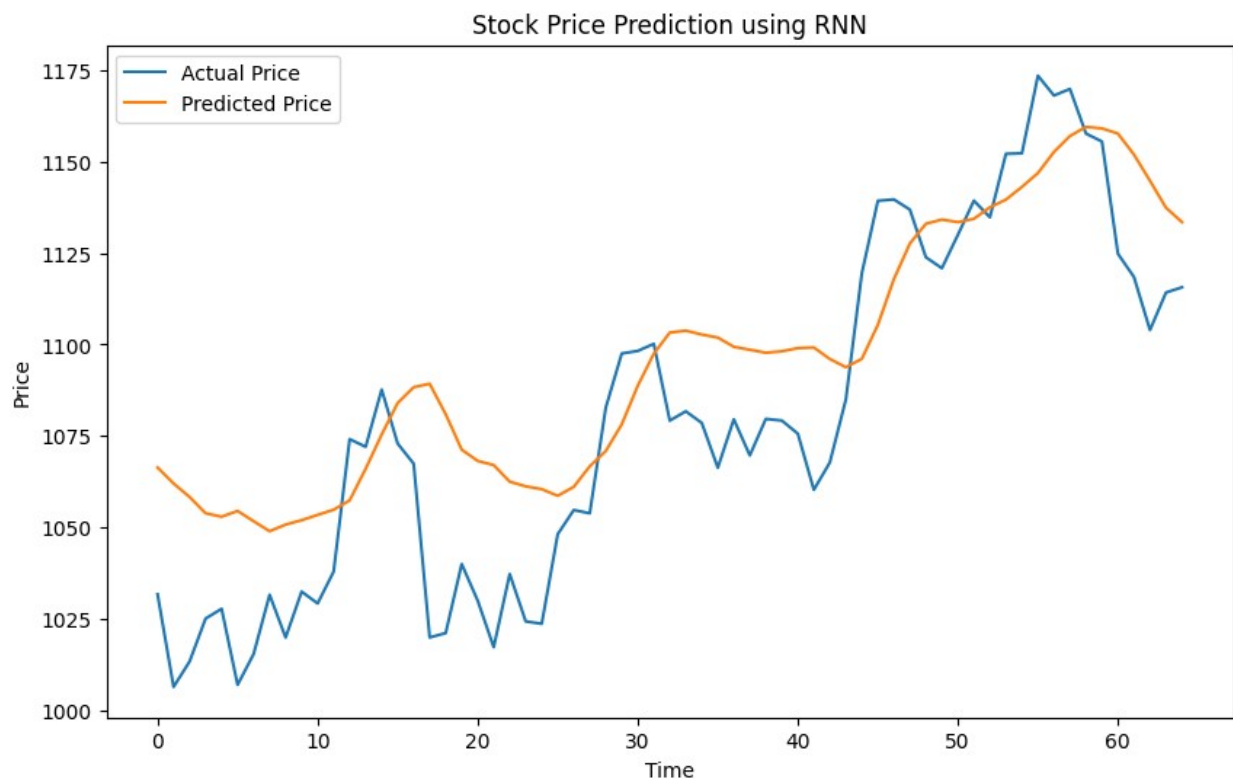
```
## Step 4: Make Predictions on Test Set
model.eval()
with torch.no_grad():
    predicted = model(x_test_tensor.to(device)).cpu().numpy()
    actual = y_test_tensor.cpu().numpy()

# Inverse transform the predictions and actual values
predicted_prices = scaler.inverse_transform(predicted)
actual_prices = scaler.inverse_transform(actual)

# Plot the predictions vs actual prices
print('Name: SARAIVANAN P')
print('Register Number: 212224230253')
plt.figure(figsize=(10, 6))
plt.plot(actual_prices, label='Actual Price')
plt.plot(predicted_prices, label='Predicted Price')
plt.xlabel('Time')
plt.ylabel('Price')
plt.title('Stock Price Prediction using RNN')
```

```
plt.legend()
plt.show()
print(f'Predicted Price: {predicted_prices[-1]}')
print(f'Actual Price: {actual_prices[-1]}')
```

Name: SARA VANAN P
Register Number: 212224230253



Predicted Price: [1133.454]
Actual Price: [1115.65]